Formal Translation Rules from WAT to Boogie

Mahmoud CHAARI

2025-2026

Contents

Chapter 1

Formal Translation Rules from WAT to Boogie

This document constitutes the formal core of our translation framework.. We define the translation from WebAssembly [1] Text (WAT) into Boogie [2], an intermediate verification language. Unlike previous work such as VeriSol [3], which translates Solidity into Boogie, our case study is more intricate: WAT is a **stack-based language**, while Boogie is **register-based in SSA form**. This semantic mismatch makes the translation significantly more challenging, as implicit data dependencies in the operand stack must be made explicit in Boogie through auxiliary procedures, global variables, helper procedures, and runtime encodings.

The chapter is structured as follows. We begin with the key challenges and notational conventions underlying our formalism. We then formalize the syntax of the WAT subset we consider, followed by the Boogie subset that serves as target language. Next, we present the principles of translation that bridge stack-based semantics and SSA-based representations. Finally, we introduce translation rules for the supported WAT instructions.

1.1 Challenges and Notational Conventions

Two major difficulties arise when defining a translation from WAT to Boogie:

- **Necessity of formal rules.** Translating a programming language into another is not a mere syntactic exercise: It amounts to defining an executable semantics in the target language. To avoid ambiguity, we must give inference-style formal rules that precisely capture how each WAT construct maps to Boogie. These rules ensure the translation is both sound and reproducible.
- **Stack versus register mismatch.** WAT is a *stack-based* language: instructions consume operands from the stack and push results implicitly. Boogie, in contrast, is *register-based and SSA-oriented*, requiring explicit variables, assignments, and control-flow. Bridging this semantic gap requires us to model the stack explicitly in Boogie using global maps and auxiliary procedures.

Notational Conventions

We use the following notations in the inference rules:

- $\vdash$ (turnstile) indicates that the translation is derived under the current typing and environment assumptions.
- $\rightarrow$ (arrow) denotes the translation of a WAT sub-expression or sub-statement into an Boogie code fragment.
- $\hookrightarrow$ (hook arrow) marks the final translation step from a WAT construct to its Boogie encoding.
- The left-hand side of a rule is always a WAT construct, while the right-hand side gives the corresponding Boogie code.

This notation, inspired by VeriSol [3], is here adapted to capture the stack-based execution semantics of WebAssembly.

1.2 Source Language: WAT

1.2.1 Motivation and Characteristics

WebAssembly (WASM) is a portable low-level format, and WAT (WebAssembly Text) is its human-readable form. For our purposes, the key characteristic is that WAT follows a **stack discipline**:

- Each instruction consumes operands from the top of the stack,
- Produces results by pushing them back,
- And relies on structured control constructs with labels.

This stack discipline makes WAT concise, but complicates translation into Boogie, where all data dependencies must be explicit.

1.2.2 Syntax of the WAT Subset

Lexical domains : We use the following metavariables throughout the grammar and the translation rules. Identifiers are used to name locals, globals, functions, memories, and tables. Labels are used by structured control-flow constructs. Natural numbers appear as indices, offsets, memory sizes, table sizes, and branch depths. String literals are used for import/export annotations.

$x, g, f, m, t \in \mathit{Identifiers}$	(names of locals, globals, functions, memories, and tables)
$L \in \mathit{Labels}$	(names of blocks, loops, or branch targets)
$i, j, n \in \mathbb{N}$	(indices, offsets, sizes, and branch depths)
$s \in \mathit{StringLiterals}$	(strings used for imports and exports)
$e \in \mathit{InitExpr}$	(constant initialization expression)

Types : WAT distinguishes between integer types (`i32`, `i64`) and floating-point types (`f32`, `f64`). These are the primitive scalar types from which all higher-level values are built.

$$
\begin{aligned}
it \in \mathit{IntType} &::= \texttt{i32} \mid \texttt{i64} \\
ft \in \mathit{FloatType} &::= \texttt{f32} \mid \texttt{f64} \\
wt \in \mathit{WAT\text{-}Types} &::= \mathit{IntType} \cup \mathit{FloatType}
\end{aligned}
$$

Expressions : We separate expressions into producers that push values onto the stack (e.g., constants, arithmetic, memory loads), and consumers/effects that pop values or change state (e.g., assignments, stores, drops). This separation clarifies which instructions contribute to dataflow and which only manipulate control or memory.

Expressions (producers)

$$
\begin{aligned}
exp_1 \in \mathit{WAT\text{-}Expressions} ::= {} & \texttt{wt.const} \mid \texttt{local.get}\ x \mid \texttt{global.get}\ x \mid exp_1\ exp_1\ op \mid exp_1\ op \\
& \mid \texttt{call}\ f \mid \texttt{call\_indirect}\ f \mid \texttt{select} \mid \texttt{memory.size} \mid \texttt{memory.grow} \\
& \mid \texttt{table.get} \mid \texttt{table.size} \mid \texttt{table.grow} \\
& \mid \texttt{it.load} \mid \texttt{it.load8\_s} \mid \texttt{it.load8\_u} \mid \texttt{it.load16\_s} \mid \texttt{it.load16\_u} \\
& \mid \texttt{i64.load32\_s} \mid \texttt{i64.load32\_u} \mid \texttt{ft.load}
\end{aligned}
$$

Expressions (consumers / effects)

$$
\begin{aligned}
exp_2 \in \mathit{WAT\text{-}Expressions} ::= {} & \texttt{local.set}\ x \mid \texttt{local.tee}\ x \mid \texttt{global.set}\ x \mid \texttt{drop} \mid \texttt{return} \\
& \mid \texttt{return\_call}\ f \mid \texttt{return\_call\_indirect} \mid \texttt{br}\ L \mid \texttt{br\_if}\ L \\
& \mid \texttt{br\_table} \mid \texttt{unreachable} \mid \texttt{memory.copy} \mid \texttt{memory.fill} \\
& \mid \texttt{table.set} \\
& \mid \texttt{it.store} \mid \texttt{it.store8} \mid \texttt{it.store16} \mid \texttt{i64.store32} \mid \texttt{ft.store}
\end{aligned}
$$

Statements : WAT statements combine expressions with control constructs such as `block`, `loop`, and `if`. Unlike typical imperative languages, WAT is structured: all branches and loops are delimited, and control transfer happens via labels instead of unstructured jumps.

$$
\begin{aligned}
wst \in \mathit{WAT\text{-}Statements} ::= {} & exp_1 \mid exp_2 \mid \texttt{block}\ L\ wst^* \mid \texttt{loop}\ L\ wst^* \\
& \mid \texttt{if (then}\ wst^*\texttt{) (else}\ wst^*\texttt{)} \mid \texttt{if}\ (wst^*)
\end{aligned}
$$

Operators : Unary and binary operators capture the usual arithmetic, comparison, and logical operations, but are parameterized by type. For example, `i32.add` and `f32.add` both denote

addition but apply to different domains.

$$
\begin{aligned}
\mathit{UnOp} ::= {} & \texttt{it.eqz} \mid \texttt{i32.wrap\_i64} \mid \texttt{it.clz} \mid \texttt{it.ctz} \mid \texttt{it.popcnt} \\
& \mid \texttt{ft.abs} \mid \texttt{ft.neg} \mid \texttt{ft.sqrt} \mid \texttt{ft.floor} \mid \texttt{ft.ceil} \mid \texttt{ft.trunc} \mid \texttt{ft.nearest} \\
& \mid \texttt{i64.extend\_i32\_s} \mid \texttt{i64.extend\_i32\_u} \\
& \mid \texttt{it.trunc\_ft\_s} \mid \texttt{it.trunc\_ft\_u} \mid \texttt{ft.convert\_it\_s} \mid \texttt{ft.convert\_it\_u} \\
& \mid \texttt{f32.demote\_f64} \mid \texttt{f64.promote\_f32}
\end{aligned}
$$

$$
\begin{aligned}
\mathit{BinOp} ::= {} & \texttt{wt.add} \mid \texttt{wt.sub} \mid \texttt{wt.mul} \mid \texttt{it.div\_s} \mid \texttt{it.div\_u} \mid \texttt{ft.div} \\
& \mid \texttt{it.rem\_s} \mid \texttt{it.rem\_u} \\
& \mid \texttt{wt.eq} \mid \texttt{wt.ne} \mid \texttt{it.lt\_s} \mid \texttt{it.le\_s} \mid \texttt{it.gt\_s} \mid \texttt{it.ge\_s} \\
& \mid \texttt{it.lt\_u} \mid \texttt{it.le\_u} \mid \texttt{it.gt\_u} \mid \texttt{it.ge\_u} \mid \texttt{ft.lt} \mid \texttt{ft.le} \mid \texttt{ft.gt} \mid \texttt{ft.ge} \\
& \mid \texttt{it.and} \mid \texttt{it.or} \mid \texttt{it.xor} \mid \texttt{it.shl} \mid \texttt{it.shr\_s} \mid \texttt{it.shr\_u} \mid \texttt{it.rotl} \mid \texttt{it.rotr} \\
& \mid \texttt{ft.min} \mid \texttt{ft.max} \mid \texttt{ft.copysign}
\end{aligned}
$$

$$op \in \mathit{UnOp} \cup \mathit{BinOp}$$

Declarations and modules : Functions, locals, and globals are declared at the module level. Imports and exports enable interoperability with the outside world. A WAT module is thus a collection of such declarations, providing the compilation and verification unit for our translation.

$$
\begin{aligned}
\mathit{FuncDecl} &::= \texttt{(func } f \texttt{ (param } x_1\ wt_1\ \ldots\ x_n\ wt_n\texttt{)(result } wt_r^*\texttt{)} \mathit{LocalDecl}^*\ wst^*\texttt{)} \\
\mathit{LocalDecl} &::= \texttt{(local } x\ wt\texttt{)} \\
\mathit{GlobalDecl} &::= \texttt{(global } x\ wt\ e\texttt{)} \mid \texttt{(global } x\ \texttt{(mut } wt\texttt{)}\ e\texttt{)} \\
\mathit{MemoryDecl} &::= \texttt{(memory } n\texttt{)} \\
\mathit{TableDecl} &::= \texttt{(table } n\ \texttt{funcref)} \\
\mathit{ImportDecl} &::= \texttt{(import } s_1\ s_2\ \texttt{(func } f\ \texttt{(param } wt^*\texttt{)(result } wt^*\texttt{)))} \\
&\quad \mid \texttt{(import } s_1\ s_2\ \texttt{(memory } n\texttt{))} \\
&\quad \mid \texttt{(import } s_1\ s_2\ \texttt{(table } n\ \texttt{funcref))} \\
\mathit{ExportDecl} &::= \texttt{(export } s\ \texttt{(func } f\texttt{))} \mid \texttt{(export } s\ \texttt{(memory } n\texttt{))} \mid \texttt{(export } s\ \texttt{(table } n\texttt{))} \\
\mathit{Module} &::= \texttt{(module } \mathit{ImportDecl}^*\ \mathit{GlobalDecl}^*\ \mathit{MemoryDecl}^*\ \mathit{TableDecl}^*\ \mathit{FuncDecl}^*\ \mathit{ExportDecl}^*\texttt{)}
\end{aligned}
$$

Here, e denotes a constant initialization expression, typically of the form `(i32.const n)`, `(i64.const n)`, `(f32.const r)`, or `(f64.const r)`. Mutable globals are represented with `(mut wt)`, while immutable globals omit the `mut` keyword.

1.2.3 Typing Function

We use a typing function $\texttt{type}(e)$ that returns the WebAssembly type of an expression e.

Given a typing environment Γ, the function inspects the syntax of e and determines the type of the value produced on the operand stack. Depending on the expression, the resulting type may be an integer (`i32`, `i64`), a floating-point value (`f32`, `f64`), or a value obtained through comparison, conversion, memory, table, or function-call operations.

Formally,

$$\Gamma \vdash e : wt$$

maps each well-typed WAT expression to the type of the value it leaves on top of the stack.

The typing environment Γ maintains the types of local variables, global variables, function parameters, function results, memories, and tables. It is used whenever the type of an expression cannot be determined syntactically.

The typing function is used throughout the translation rules to select the appropriate Boogie encoding for arithmetic operations, comparisons, conversions, memory accesses, table operations, and function invocations.

This concludes the definition of the WAT subset considered in this work. We have formalized its lexical domains, types, expressions, statements, operators, and module-level constructs. The next section introduces the target language, Boogie, and the subset of Boogie used by SafeWasm for the formal encoding of WebAssembly programs.

1.3 Target Language: Boogie

As recalled in Chapter 2, Boogie is an intermediate verification language used in several verification pipelines. Here we focus on the subset of Boogie that serves as the target of our translation.

1.3.1 Lexical domains

Boogie has a small set of primitive element types used as building blocks for more complex types:

$$bbt \in \mathit{BoogieElemTypes} ::= \texttt{int} \mid \texttt{bool} \mid \texttt{real} \mid \mathit{havoc}$$

1.3.2 Types

Complex types in Boogie include maps from one domain to another, used to model arrays, memory, or stacks:

$$bt \in \mathit{BoogieTypes} ::= bbt \mid [bbt]bt$$

Explanation. A type in Boogie is either a primitive element type (`int`, `bool`, `real`) or a mapping type $[bbt]bt$, denoting functions from elements to values. These map types are crucial for modeling memory and stack operations in our translation.

1.3.3 Expressions

Expressions represent values and computations in Boogie:

$$e \in \mathit{Exprs} ::= c \mid x \mid \mathit{op}(e_1, \ldots, e_n) \mid \mathit{uf}(e_1, \ldots, e_n) \\ \mid x[e] \mid \forall i : \mathit{bbt} :: e$$

Explanation. Expressions include constants c, variables x, applications of built-in operators op, applications of user-defined functions uf, array/mapping lookups $x[e]$, and universally quantified formulas.

1.3.4 Statements

Statements form the executable part of a Boogie program:

$$\begin{aligned} st \in \mathit{Stmts} ::= {} & \texttt{skip} \mid x := e \mid x[e] := e \mid \texttt{assume}\ e \mid \texttt{assert}\ e \\ & \mid \texttt{if}\ (e)\ \{st\}\ \texttt{else}\ \{st\} \mid st;\ st \\ & \mid \texttt{while}\ (e)\ \texttt{do}\ \{st\} \mid \texttt{call}\ \sim x := f(e_1, \ldots, e_n) \end{aligned}$$

Explanation. Boogie supports imperative-style statements: assignments, assumptions, assertions, sequencing, conditionals, loops, and procedure calls. Verification conditions are ultimately generated from `assert` statements, while `assume` allows the modeler to introduce hypotheses.

1.3.5 Declarations

A Boogie program is a collection of procedure declarations, possibly with specifications:

$$\mathit{Decl} ::= \texttt{procedure}\ f(p_1 : bt_1, \ldots, p_n : bt_n)\ \texttt{returns}\ (r_1 : bt_1, \ldots, r_k : bt_k)\ \{st\}$$

Explanation. Each procedure consists of a name, a list of typed input parameters, a list of typed return values, and a body composed of statements. Procedures are the main modular units in Boogie, and serve as the direct translation targets for WAT functions.

This concludes the presentation of the Boogie core language. We have introduced its primitive and composite types, expressions, statements, and procedures, which together form the register-based and SSA-oriented target of our translation. The contrast with WAT is clear: whereas WAT relies on an implicit stack discipline, Boogie requires explicit variables and structured control. The next section develops the principles that connect these two semantic worlds, laying out how stack-based WAT programs can be systematically mapped into verifiable Boogie code.

1.4 Translation Principles

We now explain the general principles that guide the translation from WAT to Boogie. Because WAT is a stack-based language and Boogie is register-based in SSA form, the translation must make all implicit stack operations explicit. This is achieved by modeling the operand

stack, memory, and auxiliary conversions within Boogie. The following subsections describe the simulation model used.

1.4.1 Stack Simulation

The WAT operand stack is represented explicitly in Boogie through a global stack and a stack pointer:

```
$stack : [int] real;    $sp : int;
```

The stack stores all WebAssembly values. For simplicity, every WAT value (`i32`, `i64`, `f32`, and `f64`) is represented as a Boogie value of type `real`.

Three temporary variables are used when extracting values from the stack:

```
$tmp1 : real;    $tmp2 : real;    $tmp3 : real;
```

Stack manipulation is performed through auxiliary procedures:

```
  push(val)  ⇝  $stack[$sp] := val;  $sp := $sp + 1
      pop()  ⇝  $sp := $sp - 1
popToTmp1()  ⇝  $sp := $sp - 1;  $tmp1 := $stack[$sp]
popToTmp2()  ⇝  $sp := $sp - 1;  $tmp2 := $stack[$sp]
popToTmp3()  ⇝  $sp := $sp - 1;  $tmp3 := $stack[$sp]
```

Additional helper procedures `popArgsN` are used when translating function calls. They remove the top N values from the stack and assign them to the corresponding Boogie procedure parameters.

In this way, every WAT instruction that pushes or consumes values on the operand stack can be translated into explicit Boogie statements.

1.4.2 Memory Model

WebAssembly programs manipulate a linear memory composed of pages of 64 KiB. In SafeWasm, this memory is represented by the following Boogie variables:

```
$mem : [int] int

$mem_pages : int
```

where `$mem` maps memory addresses to byte values and `$mem_pages` stores the current size of the memory measured in WebAssembly pages.

Memory accesses are implemented through auxiliary procedures that model the semantics of WebAssembly load and store instructions.

For reading memory, SafeWasm provides procedures such as

```
mem_read_u8
mem_read_s8
mem_read_u16
mem_read_s16
mem_read_u32
mem_read_s32
mem_read_u64
mem_read_s64
```

and for writing memory:

```
mem_write_u8
mem_write_u16
mem_write_u32
mem_write_u64
```

SafeWasm also supports WebAssembly bulk-memory instructions through the procedures

```
memory_grow
memory_size
memory_fill
memory_copy
```

which provide the operational semantics of memory allocation, querying, initialization, and copying.

1.4.3 Auxiliary Functions and Type Conversions

Since SafeWasm represents all WebAssembly values on the operand stack as Boogie values of type `real`, several auxiliary functions are required to bridge the gap between WebAssembly types and Boogie's type system.

Boolean conversions.

$$\texttt{bool\_to\_real}(b : \mathit{bool}) \equiv \texttt{if b then 1.0 else 0.0}$$

$$\texttt{real\_to\_bool}(r : \mathit{real}) \equiv \texttt{if r = 0.0 then false else true}$$

These functions allow Boolean values produced by comparisons to be stored on the operand stack and later recovered for conditional branching.

The following axioms establish their relationship:

$$\forall b : \mathit{bool} \,.\, \mathit{real\_to\_bool}(\mathit{bool\_to\_real}(b)) = b$$

$$\forall r : \mathit{real} \,.\, \mathit{real\_to\_bool}(r) = \mathit{false} \iff r = 0.0$$

Numeric conversions.

$$\begin{aligned} \texttt{real\_to\_int}(r : real) &: real \to int \\ \texttt{int\_to\_real}(i : int) &: int \to real \\ \texttt{bits32\_to\_real}(i : int) &: int \to real \\ \texttt{bits64\_to\_real}(i : int) &: int \to real \end{aligned}$$

These conversions are used when translating memory accesses, integer operations, floating-point instructions, and low-level WebAssembly casts.

Mathematical functions. SafeWasm models several floating-point operations through uninterpreted or axiomatized Boogie functions:

```
min_real  max_real  abs_real
sqrt_real  nearest_real
floor_real  ceil_real  trunc_real
copysign_real
```

For example, the square-root function satisfies:

$$r \geq 0 \Longrightarrow sqrt\_real(r)^2 = r$$

and

$$r \geq 0 \Longrightarrow sqrt\_real(r) \geq 0.$$

Bitwise and integer operations. Bitwise instructions are represented through dedicated Boogie functions:

```
bv_and  bv_or  bv_xor
bv_shl  bv_shr_s  bv_shr_u
bv_rotl  bv_rotr
```

Additional integer-specific operations are modeled by:

`int_rem_s`, `int_rem_u`, `int_clz`, `int_ctz`, `int_popcnt`.

These functions provide the semantic foundation for WebAssembly integer arithmetic and bit-manipulation instructions.

1.4.4 Runtime Initialization

Before executing the translated body of a WebAssembly function, SafeWasm initializes the Boogie runtime state. This initialization is performed through two auxiliary procedures: `InitRuntime` and `initGlobals`.

Explanation. The procedure `InitRuntime` initializes the execution stack and the runtime variables used by the translation, such as the stack pointer `$sp`. The procedure `initGlobals` initializes mutable global variables and memory-related runtime state according to the declarations of the original WebAssembly module. Immutable globals are instead translated as Boogie constants,constrained by axioms when their initialization value is known statically.

This initialization step ensures that the generated Boogie program starts from a runtime state consistent with the source WebAssembly module before executing the translated function body.

1.4.5 Principle of Translation

The translation follows these general principles:

- **Expressions.** Every WAT expression e is translated into a Boogie sequence that leaves its result on the stack via `push`.

$$e \rightarrow \chi \quad \text{with} \quad \chi \text{ ending in } \texttt{push(val)}$$

- **Statements.** WAT statements are translated into structured Boogie statements (`if`, `while`, `goto`), with labels used to model WAT's control-flow blocks.

$$wst \rightarrow S \quad \text{where } S \text{ is a Boogie statement/block}$$

- **Function Calls.** A WAT call pushes arguments on the stack, and the callee uses a helper procedure `popArgsN` at the start of its body to bind them to parameters.

With these principles in place, the translation rules of the next section can be read systematically: stack effects in WAT become explicit stack manipulations in Boogie, and structured control is preserved through Boogie's imperative constructs.

1.5 Formal Translation Rules

We now present the core translation rules from WAT into Boogie. For readability, we include here only representative rules from each category (constants, arithmetic, control flow, memory, and functions). The complete set of rules is deferred to Appendix. Each rule is written in inference style: the premise above the line indicates the translation of sub-expressions, and the conclusion gives the corresponding Boogie code.

1.5.1 Constants and Basic Values

$$\frac{type(n) = it}{\vdash \texttt{it.const } n \hookrightarrow \texttt{push(int\_to\_real(n))}} (\textsc{Const-int})$$

$$\frac{type(n) = ft}{\vdash \texttt{ft.const } n \hookrightarrow \texttt{push(n)}} (\textsc{Const-float})$$

Explanation. Integer constants are converted into `real` values via `int_to_real`, while floating-point constants are directly pushed onto the Boogie stack.

1.5.2 Numeric & Arithmetic Operations

$$\frac{e_1 \in exp_1 \quad e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{ll} & \chi_1; \\ & \chi_2; \\ \vdash e_1\ e_2\ \texttt{wt.add} \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(\$tmp2 + \$tmp1);} \end{array}}(\text{ADD})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{ll} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ \texttt{wt.sub} \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(\$tmp2 - \$tmp1);} \end{array}}(\text{SUB})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{ll} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ \texttt{wt.mul} \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(\$tmp2 * \$tmp1);} \end{array}}(\text{MUL})$$

$$\frac{op \in \{\texttt{div\_s}, \texttt{div\_u}\} \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{ll} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ \texttt{it.op} \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(\$tmp2 / \$tmp1);} \end{array}}(\text{DIV\_S, DIV\_U})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{ll} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ \texttt{ft.div} \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(\$tmp2 / \$tmp1);} \end{array}}(\text{DIV})$$

$$\frac{op \in \{\texttt{rem\_s}, \texttt{rem\_u}\} \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2\ \texttt{it.op} \hookrightarrow \begin{array}{l}\chi_2;\\ \chi_1;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{call push(int\_rem\_op(\$tmp2,\$tmp1));}\end{array}}\ (\text{REM\_S, REM\_U})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2\ \texttt{ft.min} \hookrightarrow \begin{array}{l}\chi_2;\\ \chi_1;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{call push(min\_real(\$tmp2,\$tmp1));}\end{array}}\ (\text{MIN})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2\ \texttt{ft.max} \hookrightarrow \begin{array}{l}\chi_2;\\ \chi_1;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{call push(max\_real(\$tmp2,\$tmp1));}\end{array}}\ (\text{MAX})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2\ \texttt{ft.copysign} \hookrightarrow \begin{array}{l}\chi_2;\\ \chi_1;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{call push(copysign\_real(\$tmp2,\$tmp1));}\end{array}}\ (\text{COPYSIGN})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\vdash \texttt{ft.abs}\ e \hookrightarrow \begin{array}{l}\chi;\\ \texttt{call popToTmp1();}\\ \texttt{call push(abs\_real(\$tmp1));}\end{array}}\ (\text{ABS})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\vdash \texttt{ft.neg}\ e \hookrightarrow \begin{array}{l}\chi;\\ \texttt{call popToTmp1();}\\ \texttt{call push(-\$tmp1);}\end{array}}\ (\text{NEG})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash \texttt{ft.sqrt}\ e \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(sqrt\_real(\$tmp1));}\end{array}}(\textsc{sqrt})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash \texttt{ft.floor}\ e \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(floor\_real(\$tmp1));}\end{array}}(\textsc{floor})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash \texttt{ft.ceil}\ e \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(ceil\_real(\$tmp1));}\end{array}}(\textsc{ceil})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash \texttt{ft.trunc}\ e \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(trunc\_real(\$tmp1));}\end{array}}(\textsc{trunc})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash \texttt{ft.nearest}\ e \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(nearest\_real(\$tmp1));}\end{array}}(\textsc{nearest})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash e\ \texttt{it.clz} \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(int\_clz(\$tmp1));}\end{array}}(\textsc{clz})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash e\ \texttt{it.ctz} \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(int\_ctz(\$tmp1));}\end{array}}(\textsc{ctz})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\chi;\ \vdash e\ \texttt{it.popcnt} \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{call push(int\_popcnt(\$tmp1));}\end{array}}(\textsc{clz})$$

1.5.3 Comparison Operations

$$\frac{e \in \mathit{exp\_1} \quad \vdash e \to \chi}{\begin{array}{l}\chi; \\ \vdash \texttt{it.eqz}\ e \hookrightarrow \texttt{call popToTmp1();} \\ \qquad\qquad \texttt{call push(bool\_to\_real(\$tmp1 == 0.0));}\end{array}}(\text{EQZ})$$

$$\frac{e_1, e_2 \in \mathit{exp}_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2; \\ \chi_1; \\ \vdash e_1\ e_2\ \texttt{wt.eq} \hookrightarrow \texttt{call popToTmp1();} \\ \qquad\qquad \texttt{call popToTmp2();} \\ \qquad\qquad \texttt{call push(bool\_to\_real(\$tmp2 == \$tmp1));}\end{array}}(\text{EQ})$$

$$\frac{e_1, e_2 \in \mathit{exp}_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2; \\ \chi_1; \\ \vdash e_1\ e_2\ \texttt{wt.ne} \hookrightarrow \texttt{call popToTmp1();} \\ \qquad\qquad \texttt{call popToTmp2();} \\ \qquad\qquad \texttt{call push(bool\_to\_real(\$tmp2 != \$tmp1));}\end{array}}(\text{NE})$$

$$\frac{\mathit{op} \in \{\texttt{it.lt\_s}, \texttt{it.lt\_u}, \texttt{ft.lt}\} \quad e_1, e_2 \in \mathit{exp}_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2; \\ \chi_1; \\ \vdash e_1\ e_2\ \mathit{op} \hookrightarrow \texttt{call popToTmp1();} \\ \qquad\qquad \texttt{call popToTmp2();} \\ \qquad\qquad \texttt{call push(bool\_to\_real(\$tmp2 < \$tmp1));}\end{array}}(\text{LT})$$

$$\frac{\mathit{op} \in \{\texttt{it.le\_s}, \texttt{it.le\_u}, \texttt{ft.le}\} \quad e_1, e_2 \in \mathit{exp}_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2; \\ \chi_1; \\ \vdash e_1\ e_2\ \mathit{op} \hookrightarrow \texttt{call popToTmp1();} \\ \qquad\qquad \texttt{call popToTmp2();} \\ \qquad\qquad \texttt{call push(bool\_to\_real(\$tmp2 <= \$tmp1));}\end{array}}(\text{LE})$$

$$\frac{\mathit{op} \in \{\texttt{it.gt\_s}, \texttt{it.gt\_u}, \texttt{ft.gt}\} \quad e_1, e_2 \in \mathit{exp}_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2; \\ \chi_1; \\ \vdash e_1\ e_2\ \mathit{op} \hookrightarrow \texttt{call popToTmp1();} \\ \qquad\qquad \texttt{call popToTmp2();} \\ \qquad\qquad \texttt{call push(bool\_to\_real(\$tmp2 > \$tmp1));}\end{array}}(\text{GT})$$

$$\frac{op \in \{\texttt{it.ge\_s}, \texttt{it.ge\_u}, \texttt{ft.ge}\} \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2;\\ \chi_1;\\ \vdash e_1\ e_2\ op \hookrightarrow \texttt{call popToTmp1();}\\ \qquad\qquad\ \texttt{call popToTmp2();}\\ \qquad\qquad\ \texttt{call push(bool\_to\_real(\$tmp2 >= \$tmp1));}\end{array}}\ (\text{GE})$$

1.5.4 Bitwise Operations

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2;\\ \chi_1;\\ \vdash e_1\ e_2\ it.and \hookrightarrow \texttt{call popToTmp1();}\\ \qquad\qquad\ \texttt{call popToTmp2();}\\ \qquad\qquad\ \texttt{call push(bv\_and(\$tmp2,\$tmp1));}\end{array}}\ (\text{AND})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2;\\ \chi_1;\\ \vdash e_1\ e_2\ it.or \hookrightarrow \texttt{call popToTmp1();}\\ \qquad\qquad\ \texttt{call popToTmp2();}\\ \qquad\qquad\ \texttt{call push(bv\_or(\$tmp2,\$tmp1));}\end{array}}\ (\text{OR})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2;\\ \chi_1;\\ \vdash e_1\ e_2\ it.xor \hookrightarrow \texttt{call popToTmp1();}\\ \qquad\qquad\ \texttt{call popToTmp2();}\\ \qquad\qquad\ \texttt{call push(bv\_xor(\$tmp2,\$tmp1));}\end{array}}\ (\text{XOR})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{l}\chi_2;\\ \chi_1;\\ \vdash e_1\ e_2\ it.shl \hookrightarrow \texttt{call popToTmp1();}\\ \qquad\qquad\ \texttt{call popToTmp2();}\\ \qquad\qquad\ \texttt{call push(bv\_shl(\$tmp2,\$tmp1));}\end{array}}\ (\text{SHL})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{rl} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ it.shr\_s \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(bv\_shr\_s(\$tmp2,\$tmp1));} \end{array}} (\text{SHR\_S})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{rl} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ it.shr\_u \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(bv\_shr\_u(\$tmp2,\$tmp1));} \end{array}} (\text{SHR\_U})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{rl} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ it.rotl \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(bv\_rotl(\$tmp2,\$tmp1));} \end{array}} (\text{ROTL})$$

$$\frac{e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\begin{array}{rl} & \chi_2; \\ & \chi_1; \\ \vdash e_1\ e_2\ it.rotr \hookrightarrow & \texttt{call popToTmp1();} \\ & \texttt{call popToTmp2();} \\ & \texttt{call push(bv\_rotr(\$tmp2,\$tmp1));} \end{array}} (\text{ROTR})$$

Explanation. Bitwise operations consume two integer operands from the WAT operand stack. SafeWasm evaluates the two operands, pops them into `$tmp1` and `$tmp2`, and delegates the corresponding bit-level semantics to dedicated Boogie functions such as `bv_and`, `bv_or`, `bv_shl`, and `bv_shr_u`. The result is then pushed back onto the stack as a `real` value.

1.5.5 Numeric Conversion Operations

WebAssembly provides several instructions to convert values between integer and floating-point types, or between values of different sizes. Since the Boogie operand stack stores only values of type `real`, these conversions are implemented through auxiliary conversion functions defined in the runtime prelude.

$$\frac{op \in \{\texttt{i64.extend\_i32\_s}, \texttt{i64.extend\_i32\_u}\} \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash e\ op \hookrightarrow \begin{array}{l}\chi;\\ \texttt{// No conversion required}\end{array}}(\textsc{Extend})$$

$$\frac{e \in exp_1 \quad \vdash e \to \chi}{\vdash e\ \texttt{i32.wrap\_i64} \hookrightarrow \begin{array}{l}\chi;\\ \texttt{// No conversion required}\end{array}}(\textsc{Wrap})$$

$$\frac{op \in \{\texttt{f64.promote\_f32}, \texttt{f32.demote\_f64}\} \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash e\ op \hookrightarrow \begin{array}{l}\chi;\\ \texttt{// No conversion required}\end{array}}(\textsc{Float-Conversion})$$

$$\frac{op \in \{\texttt{it.trunc\_ft\_s}, \texttt{it.trunc\_ft\_u}\} \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash e\ op \hookrightarrow \chi}(\textsc{Trunc})$$

$$\frac{op \in \{\texttt{ft.convert\_it\_s}, \texttt{ft.convert\_it\_u}\} \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash e\ op \hookrightarrow \chi}(\textsc{Convert})$$

1.5.6 Control Flow

$$\frac{wst \to S}{\vdash \texttt{if (then } wst\texttt{)} \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{if (real\_to\_bool(\$tmp1)) \{}\\ \quad S\\ \texttt{\}}\end{array}}(\textsc{if})$$

$$\frac{}{\vdash \texttt{br } L \hookrightarrow \texttt{goto L;}}(\textsc{br})$$

$$\frac{}{\vdash \texttt{br\_if } L \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{if (real\_to\_bool(\$tmp1)) \{}\\ \quad \texttt{goto L;}\\ \texttt{\}}\end{array}}(\textsc{br\_if})$$

$$\frac{wst \to S}{\vdash \texttt{block } L\ wst\ \texttt{end} \hookrightarrow \begin{array}{l}\texttt{label L:}\\ S\end{array}}(\textsc{block})$$

$$\frac{\vdash wst_1 \to S_1 \quad \vdash wst_2 \to S_2}{\vdash \texttt{if (then } wst_1 \texttt{) (else } wst_2 \texttt{)} \hookrightarrow \begin{array}{l}\texttt{call popToTmp1();}\\ \texttt{if (real\_to\_bool(\$tmp1)) \{}\\ \quad S_1\\ \texttt{\} else \{}\\ \quad S_2\\ \texttt{\}}\end{array}}\ (\textsc{if else})$$

$$\frac{wst \to S}{\vdash \texttt{loop } L\ wst \hookrightarrow \begin{array}{l}\texttt{label L:}\\ S\\ \texttt{goto L;}\end{array}}\ (\textsc{loop})$$

$$\frac{e_1, e_2, e_3 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2 \quad \vdash e_3 \to \chi_3}{\vdash e_1\ e_2\ e_3\ \texttt{select} \hookrightarrow \begin{array}{l}\chi_1;\\ \chi_2;\\ \chi_3;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{call popToTmp3();}\\ \texttt{if (real\_to\_bool(\$tmp1)) \{}\\ \quad \texttt{call push(\$tmp3);}\\ \texttt{\} else \{}\\ \quad \texttt{call push(\$tmp2);}\\ \texttt{\}}\end{array}}\ (\textsc{select})$$

$$\frac{}{\vdash \texttt{end} \hookrightarrow \epsilon}\ (\textsc{end})$$

$$\frac{}{\vdash \texttt{nop} \hookrightarrow \epsilon}\ (\textsc{nop})$$

Explanation. The symbol ϵ denotes the empty sequence of Boogie commands.

$$\frac{}{\vdash \texttt{drop} \hookrightarrow \texttt{pop();}}\ (\textsc{drop})$$

$$\frac{}{\vdash \texttt{unreachable} \hookrightarrow \texttt{assert false;}}\ (\textsc{unreachable})$$

Explanation. Structured control flow in WAT is mapped to explicit Boogie constructs: loops use labels and gotos, while conditionals rely on popping the top of the stack and evaluating it as a Boolean.

1.5.7 Memory Operations

$$\frac{load_i \in BoogieLocalVars}{\vdash \texttt{memory.size} \hookrightarrow \begin{array}{l}\texttt{call load\_i := memory\_size();}\\ \texttt{call push(int\_to\_real(load\_i));}\end{array}}\,(\textsc{memory.size})$$

$$\frac{load_i \in BoogieLocalVars \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash \; e\ memory.grow \hookrightarrow \begin{array}{l}\chi;\\ \texttt{call popToTmp1();}\\ \texttt{call load\_i := memory\_grow(real\_to\_int(\$tmp1));}\\ \texttt{call push(int\_to\_real(load\_i));}\end{array}}\,(\textsc{memory.grow})$$

$$\frac{idx, load_i \in BoogieLocalVars \quad load \in \{\texttt{it.load}, \texttt{ft.load}\} \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash e\ load \hookrightarrow \begin{array}{l}\chi;\\ \texttt{call popToTmp1();}\\ \texttt{idx := real\_to\_int(\$tmp1);}\\ \texttt{call load\_i := mem\_read(idx);}\\ \texttt{call push(int\_to\_real(load\_i));}\end{array}}\,(\textsc{load})$$

$$\frac{idx, load_i \in BoogieLocalVars \quad load \in \{\texttt{load8\_s/u}, \texttt{load16\_s/u}, \texttt{load32\_s/u}\} \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash e\ load \hookrightarrow \begin{array}{l}\chi;\\ \texttt{call popToTmp1();}\\ \texttt{idx := real\_to\_int(\$tmp1);}\\ \texttt{call load\_i := mem\_read\_load(idx);}\\ \texttt{call push(int\_to\_real(load\_i));}\end{array}}\,(\textsc{load-ex})$$

$$\frac{idx \in BoogieLocalVars \quad store \in \{\texttt{it.store}, \texttt{ft.store}\} \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2\ store \hookrightarrow \begin{array}{l}\chi_2;\\ \chi_1;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{idx := real\_to\_int(\$tmp2);}\\ \texttt{call mem\_write(idx, real\_to\_int(\$tmp1));}\end{array}}\,(\textsc{store})$$

$$\frac{idx \in BoogieLocalVars \quad store \in \{\texttt{store8}, \texttt{store16}, \texttt{store32}\} \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2\ store \hookrightarrow \begin{array}{l} \chi_2; \\ \chi_1; \\ \texttt{call popToTmp1();} \\ \texttt{call popToTmp2();} \\ \texttt{idx := real\_to\_int(\$tmp2);} \\ \texttt{call mem\_write(idx, real\_to\_int(\$tmp1));} \end{array}} (\text{STORE-}$$

$$\frac{e_1, e_2, e_3 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2 \quad \vdash e_3 \to \chi_3}{\vdash \texttt{memory.fill} \hookrightarrow \begin{array}{l} \chi_3; \\ \chi_2; \\ \chi_1; \\ \texttt{call popToTmp1();} \\ \texttt{call popToTmp2();} \\ \texttt{call popToTmp3();} \\ \texttt{call memory\_fill(} \\ \quad real\_to\_int(\$tmp3), \\ \quad real\_to\_int(\$tmp2), \\ \quad real\_to\_int(\$tmp1)); \end{array}} (\text{MEMORY.FILL})$$

$$\frac{e_1, e_2, e_3 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2 \quad \vdash e_3 \to \chi_3}{\vdash \texttt{memory.copy} \hookrightarrow \begin{array}{l} \chi_3; \\ \chi_2; \\ \chi_1; \\ \texttt{call popToTmp1();} \\ \texttt{call popToTmp2();} \\ \texttt{call popToTmp3();} \\ \texttt{call memory\_copy(} \\ \quad real\_to\_int(\$tmp3), \\ \quad real\_to\_int(\$tmp2), \\ \quad real\_to\_int(\$tmp1)); \end{array}} (\text{MEMORY.COPY})$$

1.5.8 Table Operations

$$\frac{idx \in BoogieLocalVars \quad e \in exp_1 \quad \vdash e \to \chi}{\vdash e \texttt{ table.get} \hookrightarrow \begin{array}{l}\chi;\\ \texttt{call popToTmp1();}\\ \texttt{idx := real\_to\_int(\$tmp1);}\\ \texttt{call \$tmp1 := table\_get(idx);}\\ \texttt{call push(\$tmp1);}\end{array}}(\text{TABLE.GET})$$

$$\frac{idx \in BoogieLocalVars \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2 \texttt{ table.set} \hookrightarrow \begin{array}{l}\chi_1;\\ \chi_2;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{idx := real\_to\_int(\$tmp2);}\\ \texttt{call table\_set(idx,\$tmp1);}\end{array}}(\text{TABLE.SET})$$

$$\frac{idx \in BoogieLocalVars}{\vdash \texttt{table.size} \hookrightarrow \begin{array}{l}\texttt{call idx := table\_size();}\\ \texttt{call push(int\_to\_real(idx));}\end{array}}(\text{TABLE.SIZE})$$

$$\frac{idx, load_i \in BoogieLocalVars \quad e_1, e_2 \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \vdash e_2 \to \chi_2}{\vdash e_1\ e_2 \texttt{ table.grow} \hookrightarrow \begin{array}{l}\chi_1;\\ \chi_2;\\ \texttt{call popToTmp1();}\\ \texttt{call popToTmp2();}\\ \texttt{idx := real\_to\_int(\$tmp1);}\\ \texttt{call load\_i := table\_grow(\$tmp2,idx);}\\ \texttt{call push(int\_to\_real(load\_i));}\end{array}}(\text{TABLE.GROW})$$

Explanation. Table instructions are translated using the Boogie runtime variables `$table:[int]real` and `$table_size : int`. The instruction `table.get` reads an entry from the table and pushes it back onto the operand stack, while `table.set` updates the table at a given index. The instruction `table.size` pushes the current table size, and `table.grow` increases the table size and returns the previous size.

1.5.9 Function Calls and Locals

$$\frac{f \in \mathit{FuncDecl} \quad e_1, \ldots, e_n \in \mathit{exp}_1 \quad \vdash e_1 \to \chi_1 \quad \cdots \quad \vdash e_n \to \chi_n}{\vdash e_1 \ \ldots \ e_n \ \texttt{call}\ f \hookrightarrow \begin{array}{l} \chi_1; \\ \ldots \\ \chi_n; \\ \texttt{call}\ f\texttt{();} \end{array}} (\textsc{Call-F})$$

$$\frac{\mathit{params}(f) = x_1, \ldots, x_n}{\vdash \texttt{func}\ f \hookrightarrow \begin{array}{l} \texttt{var arg1,...,argn : real;} \\ \texttt{var loc1,...,locm : real;} \\ \texttt{call popArgsN(arg1,...,argn);} \\ \texttt{loc1 := 0.0; ...; locm := 0.0;} \end{array}} (\textsc{Func-Prologue})$$

$$\frac{x \in \mathit{LocalVars}}{\vdash \texttt{local.get}\ x \hookrightarrow \texttt{call push(}x\texttt{);}} (\textsc{Local-Get})$$

$$\frac{x \in \mathit{LocalVars} \quad e \in \mathit{exp}_1 \quad \vdash e \to \chi}{\vdash e\ \texttt{local.set}\ x \hookrightarrow \begin{array}{l} \chi; \\ \texttt{call popArgs1(}x\texttt{);} \end{array}} (\textsc{Local-Set})$$

$$\frac{x \in \mathit{LocalVars} \quad e \in \mathit{exp}_1 \quad \vdash e \to \chi}{\vdash e\ \texttt{local.tee}\ x \hookrightarrow \begin{array}{l} \chi; \\ \texttt{call popArgs1(}x\texttt{);} \\ \texttt{call push(}x\texttt{);} \end{array}} (\textsc{Local-Tee})$$

$$\frac{}{\vdash \texttt{return} \hookrightarrow \texttt{goto func\_exit;}} (\textsc{return})$$

Explanation. Function calls evaluate arguments left-to-right, push them, then call the Boogie procedure. Locals are assigned by popping values from the stack via auxiliary `popArgsN` procedures.

1.5.10 Indirect and Tail Calls

Unlike direct calls, indirect calls obtain the callee dynamically from a WebAssembly table. Since the target procedure cannot be resolved statically, SafeWasm conservatively models indirect calls by discarding the arguments and the table index, then pushing nondeterministic return values whenever the callee is expected to return results.

$$\frac{e_1, \ldots, e_n, e_i \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \cdots \quad \vdash e_n \to \chi_n \quad \vdash e_i \to \chi_i}{\vdash e_1 \cdots e_n\ e_i\ \texttt{call\_indirect} \hookrightarrow \begin{array}{l}\chi_1;\\ \cdots\\ \chi_n;\\ \chi_i;\\ \texttt{call popDiscard}_{n+1}\texttt{();}\\ \texttt{for each expected result do}\\ \quad \texttt{call push(nd\_real());}\end{array}} (\textsc{Call-Indirect})$$

$$\frac{f \in \mathit{FuncDecl} \quad e_1, \ldots, e_n \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \cdots \quad \vdash e_n \to \chi_n}{\vdash e_1 \cdots e_n\ \texttt{return\_call}\ f \hookrightarrow \begin{array}{l}\chi_1;\\ \cdots\\ \chi_n;\\ \texttt{call } f\texttt{();}\\ \texttt{goto func\_exit;}\end{array}} (\textsc{Return-Call})$$

$$\frac{e_1, \ldots, e_n, e_i \in exp_1 \quad \vdash e_1 \to \chi_1 \quad \cdots \quad \vdash e_n \to \chi_n \quad \vdash e_i \to \chi_i}{\vdash e_1 \cdots e_n\ e_i\ \texttt{return\_call\_indirect} \hookrightarrow \begin{array}{l}\chi_1;\\ \cdots\\ \chi_n;\\ \chi_i;\\ \texttt{call popDiscard}_{n+1}\texttt{();}\\ \texttt{for each expected result do}\\ \quad \texttt{call push(nd\_real());}\\ \texttt{goto func\_exit;}\end{array}} (\textsc{Return-Call-Indirect})$$

Explanation. Direct tail calls are translated as ordinary Boogie procedure calls followed by a jump to the function exit label. In contrast, indirect calls cannot be resolved statically because the callee is obtained from a WebAssembly table. SafeWasm therefore adopts a conservative abstraction: all arguments together with the table index are discarded using the auxiliary `popDiscardN` procedure, and each expected return value is modeled by a fresh nondeterministic value produced by `nd_real()` and pushed onto the operand stack. This abstraction preserves soundness while avoiding explicit reasoning about dynamic function tables.

1.5.11 Global Variables

$$\frac{g \in \mathit{GlobalVars}}{\vdash \texttt{global.get}\ g \hookrightarrow \texttt{call push(}g\texttt{);}}\ (\textsc{Global-Get})$$

$$\frac{g \in \mathit{GlobalVars} \quad e \in \mathit{exp}_1 \quad \vdash e \to \chi}{\begin{array}{l}\vdash e\ \texttt{global.set}\ g \hookrightarrow \chi\texttt{;} \\ \qquad\qquad\quad \texttt{call popToTmp1();} \\ \qquad\qquad\quad \texttt{g := \$tmp1;}\end{array}}\ (\textsc{Global-Set})$$

1.5.12 Imported Functions

Imported WebAssembly functions have no implementation within the translated module. Therefore, SafeWasm translates each imported function into a Boogie stub procedure with the same signature. The procedure models an arbitrary external behavior using nondeterministic values.

$$\frac{f \in \mathit{ImportDecl} \quad \texttt{returns}(f) = \epsilon}{\begin{array}{l}\vdash \texttt{(import}\ \cdots\ \texttt{(func}\ f\texttt{))} \hookrightarrow \texttt{procedure}\ \mathit{fimport}_i\texttt{(...);} \\ \qquad\qquad\qquad\qquad\quad \texttt{// no value is pushed}\end{array}}\ (\textsc{Import-Void})$$

$$\frac{f \in \mathit{ImportDecl} \quad \texttt{returns}(f) = wt}{\begin{array}{l}\vdash \texttt{(import}\ \cdots\ \texttt{(func}\ f\texttt{))} \hookrightarrow \texttt{procedure}\ \mathit{fimport}_i\texttt{(...);} \\ \qquad\qquad\qquad\qquad\quad \texttt{havoc result;} \\ \qquad\qquad\qquad\qquad\quad \texttt{call push(result);}\end{array}}\ (\textsc{Import-Return})$$

Explanation. Imported functions are translated into Boogie stub procedures because their implementation is not available inside the translated module. If an imported function has no return value, the generated stub only models the external call and does not modify the operand stack. If it returns a value, SafeWasm nondeterministically generates a result using `havoc` and pushes it onto the stack. This provides a conservative abstraction of external behavior.

1.5.13 Global Declarations

Each WebAssembly global declaration is translated into a Boogie global variable. During module initialization, the initialization expression is evaluated and its resulting value is assigned to the corresponding global.

$$\frac{g \in \mathit{GlobalDecl} \quad \vdash e \to \chi}{\begin{array}{l}\texttt{var}\ g\texttt{:}\ \ \texttt{real;} \\ \vdash \texttt{(global}\ g\ wt\ e\texttt{)} \hookrightarrow \chi\texttt{;} \\ \qquad\qquad\quad \texttt{call popArgs1(}g\texttt{);}\end{array}}\ (\textsc{Global})$$

Explanation. Each global variable is represented as a Boogie global of type `real`. The initialization expression is translated and evaluated during module initialization, after which its value is assigned to the generated Boogie variable.

1.5.14 Exports

Export declarations only expose functions or globals to the execution environment and do not modify the operational semantics of the program. Consequently, they are ignored during the translation.

$$\frac{e \in \mathit{ExportDecl}}{\vdash \texttt{(export } \cdots\texttt{)} \hookrightarrow \epsilon}(\textsc{Export})$$

Explanation. Export declarations have no effect on the verification semantics. Therefore, SafeWasm does not generate any Boogie code for exports. Only the selected entry procedure used for verification is preserved.

“‘latex

1.6 Discussion and Scope

The formal translation rules presented in this chapter constitute the core specification of the SafeWasm translation framework. They directly address the two fundamental challenges identified at the beginning of this chapter:

1. the **need for formal inference rules**, ensuring that the translation is precise, unambiguous, reproducible, and suitable for reasoning about correctness;
2. the **semantic mismatch between WAT and Boogie**. While WAT is a stack-based language in which instructions implicitly consume and produce operands on the execution stack, Boogie follows a register-based, SSA-oriented programming model. Bridging this gap required the introduction of an explicit stack simulation together with auxiliary procedures for stack manipulation, numeric conversions, memory accesses, table operations, and the conservative modeling of imported and indirect function calls.

The rules presented in this chapter cover all instruction categories currently supported by SafeWasm, including constants, arithmetic and comparison operations, bitwise and numeric conversion instructions, memory and table operations, control-flow constructs, function calls, local and global variable manipulation, imported functions, and module-level declarations. Together, these rules formally specify how the implicit stack semantics of WebAssembly are translated into explicit Boogie code. Compared to previous work such as VeriSol [3], which translates Solidity into Boogie, our formalization must additionally capture the stack-based execution semantics of WebAssembly. This required the design of an explicit runtime model together with carefully defined helper procedures and auxiliary functions that faithfully preserve the operational behavior of WAT during translation. These aspects constitute one of the main contributions of this work

Bibliography

[1] Mozilla Developer Network, "Webassembly documentation," https://developer.mozilla.org/en-US/docs/WebAssembly, 2024, accessed: 09/2025.

[2] K. R. M. Leino, "This is boogie 2," Microsoft Research, Tech. Rep., 2008. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2016/12/krml178.pdf

[3] S. K. Lahiri, S. Qadeer, and E. Jackson, "Formal verification of smart contracts with verisol," in *Verified Software: Theories, Tools, and Experiments (VSTTE)*, 2019. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2019/09/vstte_camera_ready.pdf